Практиечская работа №4

Общее описание

Тема: Client-Side Prediction, Server Reconciliation и наблюдение сетевой синхронизации в Unreal Engine.

Цель: создать многопользовательский прототип на Unreal Engine с серверно-авторитетным перемещением персонажа, зафиксировать влияние задержки на управление и показать, как встроенный `CharacterMovementComponent` компенсирует её за счёт локального предсказания и серверных коррекций.

Результат: запускаемый сетевой прототип минимум для двух игроков, работающий по модели «сервер — авторитет», видеосравнение поведения при задержке с предсказанием и без него, а также документ `docs/Prediction_Implementation.md` .

Термины, которые нужно понимать

- **Server authority** — сервер принимает окончательное решение о допустимом состоянии игрового мира; клиент не является доверенным источником позиции игрока.
- **Autonomous Proxy** — экземпляр персонажа, которым управляет локальный игрок на клиенте.
- **Simulated Proxy** — копия чужого персонажа на клиенте, получающая сетевые обновления.
- **Client-Side Prediction (CSP)** — немедленное локальное применение ввода без ожидания ответа сервера.
- **Server Reconciliation** — сверка предсказанного состояния с авторитетным состоянием сервера, исправление расхождения и повторное применение ещё не подтверждённых вводов.
- **Saved Move** — сохранённый локальный ввод/шаг движения, необходимый для повторного проигрывания после коррекции.
- **Network smoothing** — визуальное сглаживание перемещения удалённых персонажей и корректировок, не отменяющее серверную авторитетность.

Задание

Разработайте в Unreal Engine сетевой прототип с двумя и более игроками. Каждый игрок должен управлять персонажем с перемещением, а сервер — оставаться авторитетным источником состояния.

1. Создайте проект Unreal Engine на основе шаблона **Third Person** на C++ или Blueprint.
2. Настройте запуск в режиме **Listen Server** и подключение как минимум одного клиента:
   - в Play In Editor выберите число игроков `2`;
   - задайте режим `Play As Client` либо запустите отдельный клиент;
   - убедитесь, что оба игрока видят друг друга и могут перемещаться.
3. Используйте `ACharacter` и `UCharacterMovementComponent` для перемещения. Не заменяйте авторитетную проверку сервера прямой репликацией клиентской позиции.
4. Реализуйте действие, чувствительное к задержке: кроме ходьбы и поворота добавьте хотя бы одно из следующих действий:
   - рывок (dash);
   - прыжок с ограничением повторного использования;
   - спринт;
   - изменение стойки/приседание;
   - простая способность с кулдауном.
5. Для изменяемого действия определите, какие данные должны быть проверены на сервере: например, наличие персонажа на земле, остаток выносливости, время последнего рывка, кулдаун, направление движения. Сервер обязан отклонять недопустимое действие.
6. Проведите базовый эксперимент **без локального предсказания пользовательского действия**:
   - сделайте вариант, где действие запускается только после серверного RPC и видимого ответа сервера;
   - включите эмуляцию задержки не менее `100 мс` в одном направлении;
   - зафиксируйте визуально или в логе задержку между нажатием клавиши и реакцией персонажа.
7. Реализуйте вариант **с клиентским предсказанием**:
   - локально сразу показывайте допустимый ожидаемый результат действия;
   - отправляйте на сервер не итоговую позицию, а намерение/ввод: направление, флаги состояния, временную метку или порядковый номер — в зависимости от реализации;

- сервер самостоятельно проверяет и применяет действие;
- при расхождении клиент получает авторитетное состояние, исправляет себя и применяет сохранённые неподтверждённые вводы повторно.

8. Для обычного перемещения используйте штатный сетевой механизм `CharacterMovementComponent` : он уже реализует отправку сохранённых ходов, серверную проверку, коррекции и повторное проигрывание. При расширении перемещения включите свою логику в этот механизм либо обоснуйте отдельный безопасный RPC-путь.
9. Включите эмуляцию сети и протестируйте минимум два профиля:
   - `100 мс` задержки без потерь;
   - `150–200 мс` задержки и `3–5%` потерь пакетов.
10. Подготовьте видео или серию подписанных скриншотов, показывающих один и тот же сценарий:

- без локального предсказания;
- с предсказанием;
- после серверной коррекции при намеренно созданном расхождении.

Минимальный сценарий демонстрации

1. Запустить Listen Server и клиент.
2. На клиенте показать перемещение и выбранное игровое действие при задержке.
3. Продемонстрировать, что сервер видит и реплицирует подтверждённое состояние.
4. Искусственно вызвать расхождение: например, выполнить рывок у препятствия, попытаться активировать действие до завершения кулдауна или временно изменить параметр на клиенте.
5. Показать, что сервер не принимает недопустимое состояние, а клиент возвращается к авторитетному результату без зависания или разрыва соединения.

Технические ориентиры

Рекомендуемая базовая архитектура

Компонент	Ответственность
`GameMode` (только сервер)	Правила матча, создание игроков, авторитетные решения.

`PlayerController`	Сбор локального ввода, владение Pawn, вызов RPC для дискретных действий.
`Character`	Состояние персонажа, реплицируемые свойства, реализация игровых действий.
`CharacterMovementComponent`	Сетевая обработка движения, сохранение ходов, отправка ввода, серверная проверка и коррекция.
Сервер	Валидация правил: кулдауны, коллизии, состояние персонажа, допустимость действия.
Клиент-владелец	Немедленная обратная связь и предсказание; принятие серверных корректировок.
Удалённые клиенты	Отображение состояния через репликацию и сглаживание.

Эмуляция сети

Используйте встроенную эмуляцию Unreal Engine через меню Play → Advanced Settings → Network Emulation либо консольные команды. Допустимы, например:

```
NetEmulation.PktLag 100
NetEmulation.PktLoss 0
```

или профиль с входящей/исходящей задержкой:

```
NetEmulation.PktLag 150
NetEmulation.PktLagVariance 25
NetEmulation.PktLoss 5
```

Перед чистовой демонстрацией отключите эмуляцию или перезапустите сессию. Конкретный набор доступных параметров зависит от версии движка; в отчёте укажите версию Unreal Engine и фактически применённые команды/настройки.

Важное ограничение

Не засчитывается решение, в котором клиент передаёт серверу готовые координаты и сервер без проверки записывает их в `ActorLocation`. Такая схема позволяет телепортацию и не демонстрирует reconciliation. Клиент должен передавать ввод или запрос действия; сервер должен вычислять или проверять результат.

Требования к репозиторию GitVerse

Создайте рабочую ветку `feature/prediction` от актуальной ветки `main`. После завершения оформите Pull Request в `main`.

В репозитории должны присутствовать:

```
README.md
docs/
  Prediction_Implementation.md
  Prediction_Test_Matrix.md
  media/
    baseline_no_prediction.*
    prediction_enabled.*
    correction_example.*
Source/ или Content/
```

Допускается хранить видео во внешнем облачном хранилище, если в `docs/Prediction_Implementation.md` есть проверяемая ссылка, описание ролика и время ключевого момента. Скриншоты, диаграммы и текстовые файлы хранятся в GitVerse.

Содержание `docs/Prediction_Implementation.md`

Документ должен содержать:

1. Название проекта, состав команды, версию Unreal Engine, хеш коммита для демонстрации.
2. Диаграмму или таблицу потока данных: ввод клиента → локальное предсказание → серверная проверка → репликация/коррекция → повторное применение ввода.
3. Описание ролей `GameMode`, `PlayerController`, `Character` и `CharacterMovementComponent` в проекте.
4. Описание выбранного действия и перечень серверных проверок.
5. Сравнение двух вариантов: без локального предсказания и с ним.
6. Описание случая расхождения и результата серверной коррекции.
7. Ссылки на код, скриншоты/видео и известные ограничения реализации.

Содержание `docs/Prediction_Test_Matrix.md`

Тест	Сетевые условия	Действие	Ожидаемый результат	Фактический результат	Артефакт

T1	Без эмуляции	Движение	Плавное перемещение и репликация	Заполняется студентом	Ссылка
T2	100 мс, 0% loss	Действие без CSP	Видимая задержка до подтверждения	Заполняется студентом	Ссылка
T3	100 мс, 0% loss	Действие с CSP	Немедленный отклик, сервер сохраняет авторитетность	Заполняется студентом	Ссылка
T4	150–200 мс, 3–5% loss	Движение и действие	Система не принимает недопустимое состояние; возможны коррекции	Заполняется студентом	Ссылка
T5	150–200 мс, 3–5% loss	Намеренное расхождение	Клиент получает авторитетную коррекцию	Заполняется студентом	Ссылка

Практическая работа принимается при очной демонстрации. Каждый студент должен объяснить используемые термины, показать соответствующий фрагмент кода и ответить, почему сервер не должен безусловно доверять клиентской позиции.

Сопровождающие материалы

- Epic Games Documentation — **Networked Movement in the Character Movement Component**: модель Autonomous Proxy, `SavedMoves`, обработка `ServerMove`, коррекции `ClientAdjustPosition` и повторное проигрывание ввода. https://dev.epicgames.com/documentation/en-us/unreal-engine/understanding-networked-movement-in-the-character-movement-component-for-unreal-engine
- Epic Games Documentation — **Using Network Emulation in Unreal Engine**: эмуляция задержки и потерь, параметры `PktLag`, `PktLoss`, `PktIncomingLag`. https://dev.epicgames.com/documentation/en-us/unreal-engine/using-network-emulation-in-unreal-engine

- Epic Games Documentation — **Console Commands for Network Debugging**: команды и параметры сетевой отладки. https://dev.epicgames.com/documentation/en-us/unreal-engine/console-commands-for-network-debugging-in-unreal-engine
- Epic Games Documentation — **Comparing Mover and Character Movement Component**: сравнение модели ввода, серверной проверки и коррекции в системах движения Unreal Engine. https://dev.epicgames.com/documentation/en-us/unreal-engine/comparing-mover-and-character-movement-component-in-unreal-engine

---

Итог

После выполнения практической работы студент умеет запускать и тестировать сетевой прототип в Unreal Engine, понимает границы ответственности клиента и сервера, способен объяснить назначение client-side prediction и server reconciliation, а также может воспроизводимо оценивать влияние задержки и потерь на игровой отклик.